



Programação
Orientada a
Objetos

Prof. Dr.
Bento Rafael
Siqueira

Introdução aos
Relacionamen-
tos

Associação

Agregação

Composição

Comparação
dos Relaciona-
mentos

Implementação
em C#

Exemplos de
Código

Exercícios
Práticos

Revisão

Programação Orientada a Objetos

GCT052 - Aula 2.5 - Relacionamentos entre Objetos

Prof. Dr. Bento Rafael Siqueira

Universidade Federal de Lavras
Departamento de Ciência da Computação

8 de outubro de 2025

Sumário

- 1 Introdução aos Relacionamentos
- 2 Associação
- 3 Agregação
- 4 Composição
- 5 Comparação dos Relacionamentos
- 6 Implementação em C#
- 7 Exemplos de Código
- 8 Exercícios Práticos
- 9 Boas Práticas
- 10 Próximos Passos

O que são Relacionamentos entre Objetos?

- **Relacionamentos** definem como objetos se conectam e interagem
- **Associação**: Relacionamento simples entre objetos independentes
- **Agregação**: Relacionamento "tem um" onde o objeto parte pode existir independentemente
- **Composição**: Relacionamento "tem um" onde o objeto parte não pode existir sem o todo

Analogia do Mundo Real:

- **Associação**: Pessoa e Livro (pessoa pode ler livro)
- **Agregação**: Universidade e Professor (professor pode existir sem universidade)
- **Composição**: Casa e Quarto (quarto não existe sem casa)

Por que estudar Relacionamentos?

- 1 **Modelagem Realista:** Representa melhor o mundo real
- 2 **Reutilização:** Objetos podem ser reutilizados em diferentes contextos
- 3 **Flexibilidade:** Facilita mudanças e manutenção
- 4 **Organização:** Código mais estruturado e compreensível
- 5 **Design Patterns:** Base para padrões de projeto avançados

Vantagens:

- Código mais modular e flexível
- Facilita trabalho em equipe
- Melhora manutenibilidade
- Permite modelagem complexa

O que é Associação?

- **Associação:** Relacionamento mais fraco entre objetos independentes
- **Características:**
 - Objetos podem existir independentemente
 - Relacionamento temporário ou permanente
 - Ambos os objetos têm vida própria
 - Comunicação através de métodos

Exemplos do Dia a Dia:

- **Pessoa e Carro** (pessoa pode dirigir carro)
- **Professor e Aluno** (professor ensina aluno)
- **Cliente e Produto** (cliente compra produto)

Tipos de Associação

- 1 **Associação Simples:** Relacionamento básico entre duas classes
- 2 **Associação Unidirecional:** Um objeto conhece o outro, mas não vice-versa
- 3 **Associação Bidirecional:** Ambos os objetos se conhecem mutuamente
- 4 **Associação Reflexiva:** Objeto se relaciona consigo mesmo

Multiplicidade:

- **1:1:** Um para um
- **1:N:** Um para muitos
- **N:N:** Muitos para muitos

O que é Agregação?

- **Agregação:** Relacionamento "tem um" onde a parte pode existir independentemente
- **Características:**
 - Objeto todo contém objetos partes
 - Partes podem existir sem o todo
 - Relacionamento de propriedade fraca
 - Destruição do todo não destrói as partes

Exemplos do Mundo Real:

- **Universidade e Professor** (universidade tem professores)
- **Carro e Roda** (carro tem rodas)
- **Escola e Aluno** (escola tem alunos)

Características da Agregação

- **Símbolo:** Losango vazio na UML
- **Propriedade:** "tem um" ou "contém"
- **Vida Útil:** Partes podem sobreviver ao todo
- **Criação:** Todo pode criar ou receber partes
- **Destruição:** Todo pode ser destruído sem afetar partes

Implementação em C#:

- Uso de propriedades ou campos
- Injeção de dependência
- Coleções para múltiplas partes
- Métodos para gerenciar partes

O que é Composição?

- **Composição:** Relacionamento "tem um" onde a parte não pode existir sem o todo
- **Características:**
 - Objeto todo possui objetos partes
 - Partes não podem existir independentemente
 - Relacionamento de propriedade forte
 - Destruição do todo destrói as partes

Exemplos do Mundo Real:

- **Casa e Quarto** (casa tem quartos)
- **Carro e Motor** (carro tem motor)
- **Pessoa e Coração** (pessoa tem coração)

Características da Composição

- **Símbolo:** Losango preenchido na UML
- **Propriedade:** "possui" ou "é composto por"
- **Vida Útil:** Partes dependem do todo
- **Criação:** Todo cria suas próprias partes
- **Destruição:** Todo destrói suas partes ao ser destruído

Implementação em C#:

- Criação de objetos no construtor
- Propriedades somente leitura
- Gerenciamento de ciclo de vida
- Métodos para operações nas partes

Diferenças entre os Relacionamentos

Característica	Associação	Agregação	Composição
Força do relacionamento	Fraca	Média	Forte
Existência independente	Sim	Sim	Não
Propriedade	Não	Parcial	Total
Ciclo de vida	Independente	Independente	Dependente
Símbolo UML	Linha simples	Losango vazio	Losango preenchido

Regra Prática:

- **Associação:** "usa" ou "conhece"
- **Agregação:** "tem" (pode existir sem)
- **Composição:** "possui" (não pode existir sem)

Exemplos Práticos de Comparação

Cenário: Sistema de Biblioteca

- **Associação:** Bibliotecario e Livro
 - Bibliotecário pode trabalhar com diferentes livros
 - Livro pode ser usado por diferentes bibliotecários
- **Agregação:** Biblioteca e Livro
 - Biblioteca contém livros
 - Livro pode existir fora da biblioteca
- **Composição:** Livro e Página
 - Livro é composto por páginas
 - Página não pode existir sem o livro

Estrutura Básica para Relacionamentos

Programação
Orientada a
Objetos

Prof. Dr.
Bento Rafael
Siqueira

Introdução aos
Relacionamen-
tos

Associação

Agregação

Composição

Comparação
dos Relaciona-
mentos

Implementação
em C#

Exemplos de
Código

Exercícios
Práticos

Elementos principais:

- **Classes independentes** para associação
- **Propriedades** para agregação e composição
- **Construtores** para inicialização
- **Métodos** para operações relacionadas
- **Collections** para relacionamentos 1:N

Exemplo básico: Ver arquivos Exemplos/

Padrões de Implementação

Associação:

- Referências através de métodos
- Parâmetros em métodos
- Propriedades opcionais

Agregação:

- Propriedades públicas/privadas
- Injeção via construtor
- Coleções para múltiplas partes

Composição:

- Criação no construtor
- Propriedades somente leitura
- Gerenciamento de ciclo de vida

Exemplo 1: Sistema de Biblioteca - Associação

Arquivo: Exemplos/Bibliotecario.cs e Livro.cs

- Classe Bibliotecario com método ProcessarLivro()
- Classe Livro independente
- Relacionamento através de parâmetros de método
- Ambos podem existir independentemente

Características:

- Relacionamento temporário
- Comunicação via métodos
- Flexibilidade máxima

Exemplo 2: Sistema de Universidade - Agregação

Arquivo: Exemplos/Universidade.cs e Professor.cs

- Classe Universidade com lista de Professor
- Classe Professor independente
- Métodos para adicionar/remover professores
- Professores podem existir sem universidade

Características:

- Propriedade fraca
- Ciclo de vida independente
- Flexibilidade moderada

Exemplo 3: Sistema de Casa - Composição

Arquivo: Exemplos/Casa.cs e Quarto.cs

- Classe Casa que cria seus próprios Quartos
- Classe Quarto dependente da casa
- Quartos são criados no construtor da casa
- Quartos não podem existir sem casa

Características:

- Propriedade forte
- Ciclo de vida dependente
- Controle total sobre partes

Exercício 1: Sistema de Loja

Crie um sistema de loja com:

Classes necessárias:

- Loja (nome, endereço)
- Produto (nome, preço, estoque)
- Vendedor (nome, comissão)
- Cliente (nome, telefone)

Relacionamentos:

- **Associação:** Cliente compra Produto
- **Agregação:** Loja tem Vendedores e Produtos
- **Composição:** Loja tem Caixa (não pode existir sem loja)

Tempo: 25 minutos

Exercício 2: Sistema de Escola

Crie um sistema escolar com:

Classes necessárias:

- Escola (nome, endereço)
- Professor (nome, disciplina)
- Aluno (nome, matrícula)
- Disciplina (nome, carga horária)

Relacionamentos:

- **Associação:** Professor ensina Disciplina
- **Agregação:** Escola tem Professores e Alunos
- **Composição:** Escola tem Direção (não existe sem escola)

Tempo: 30 minutos

Exercício 3: Sistema de Hospital

Crie um sistema hospitalar com:

Classes necessárias:

- Hospital (nome, especialidades)
- Medico (nome, CRM, especialidade)
- Paciente (nome, idade, sintomas)
- Consulta (data, diagnóstico)

Relacionamentos:

- **Associação:** Médico atende Paciente
- **Agregação:** Hospital tem Médicos
- **Composição:** Hospital tem Pronto Socorro

Tempo: 35 minutos

Design de Relacionamentos

- **Escolha o relacionamento correto** baseado no domínio
- **Use composição** quando a parte não pode existir sem o todo
- **Use agregação** quando a parte pode existir independentemente
- **Use associação** para relacionamentos temporários ou de uso

Princípios:

- **Princípio da Responsabilidade Única:** Cada classe tem uma responsabilidade
- **Princípio Aberto/Fechado:** Aberto para extensão, fechado para modificação
- **Princípio da Inversão de Dependência:** Dependenda de abstrações, não de concretizações

Implementação em C#

- **Use properties** para relacionamentos simples
- **Use collections** para relacionamentos 1:N
- **Implemente métodos** para operações relacionadas
- **Use construtores** apropriados para cada tipo
- **Documente** os relacionamentos com XML comments

Exemplo de XML Comments:

- `/// <summary>`
- `/// Relacionamento de agregação com professores`
- `/// </summary>`

Testes e Validação

- **Teste a criação** de objetos relacionados
- **Valide a destruição** apropriada
- **Verifique a independência** quando aplicável
- **Teste operações** em relacionamentos

Validações importantes:

- Objetos não podem ser nulos quando necessário
- Relacionamentos devem ser consistentes
- Ciclo de vida deve ser respeitado
- Operações devem manter integridade

Tópicos Relacionados

1 Herança

- Relacionamento "é um"
- Reutilização de código
- Polimorfismo

2 Interfaces

- Contratos de comportamento
- Múltiplas implementações
- Abstração avançada

3 Design Patterns

- Padrões de relacionamento
- Soluções reutilizáveis
- Arquitetura de software

Leitura Recomendada

- **Microsoft Docs:** Object-Oriented Programming in C#
- **Clean Code:** Robert C. Martin
- **Design Patterns:** Gang of Four
- **UML Distilled:** Martin Fowler

Prática Recomendada:

- 1 Implemente os exemplos da pasta Exemplos
- 2 Resolva os exercícios propostos
- 3 Crie seus próprios sistemas com relacionamentos
- 4 Pratique identificando relacionamentos no mundo real

Dúvidas e Discussão

Obrigado pela atenção!

Pontos-Chave da Aula:

- 1 **Associação** é o relacionamento mais fraco entre objetos independentes
- 2 **Agregação** permite que partes existam independentemente do todo
- 3 **Composição** cria dependência forte entre todo e partes
- 4 **Escolha correta** do relacionamento é fundamental para um bom design